Migrating Off Dropbox: Beyond Dotfiles

Ronald G. Thomas

2026-05-07



Figure 1: A wooden tray on a plain workbench, with one large compartment on the left holding a single round glass weight and three narrower compartments on the right holding three different small objects, suggesting a single category being separated into three purpose-fit ones.

One sync mechanism for everything becomes several mechanisms, each matched to what it carries.

Introduction

For years, one Dropbox folder held nearly everything in this research workflow: shell and editor configuration, about 19 GB of project repositories, personal documents, shared LaTeX and R utilities, and, through a handful of symbolic links, several application settings folders. Leaving it took longer than copying files, because the difficult part was deciding what each category of file needed, and because finishing the job surfaced problems that a sync daemon had kept out of sight. This post records where everything went, what the move exposed, and the procedures that made the final deletion safe.

> **Note**
>
> Part 4 of the security series; start with Security Foundations for a Multi-Machine Research Fleet for the overview.

The fleet involved is the one described in the series hub: three Macs (julia, athena, joe), a Linux home server (mercury) and a Linux laptop (zeus). Dropbox ran on the three Macs and, as it turned out, on zeus as well. It is now uninstalled from every machine.

Motivations

- **A new machine should not depend on a sync client.** Before the move, a freshly installed Mac with the dotfiles deployed still could not open a project until Dropbox had signed in and downloaded several gigabytes.
- **Append-only files and whole-file sync do not mix.** Shell history and an AI assistant's prompt log were written by several machines through a synced link; every overlapping write produced a silent conflict copy.
- **A synced folder is an exposure surface.** Anything a tool writes into a linked folder leaves the machine, whether or not anyone intended it to.
- **Deletion in a shared account is not local.** Every device linked to the account can propagate, or reverse, a deletion.
- **Derived artifacts should not travel.** Package libraries, caches and model output inflate a synced tree and make exclusion rules fragile.

Objectives

By the end of this post the reader should be able to:

1. Classify the contents of a synced folder by how each category should be distributed: by git, by backup only, or not at all.
2. Recognize settings folders linked into a synced tree as a credential exposure, and respond with rotation rather than reassurance.
3. Delete a shared copy in an order that no linked device can undo.
4. Audit `$HOME` for links into the synced tree before uninstalling the client.
5. Keep several machines in step afterwards without any sync daemon.

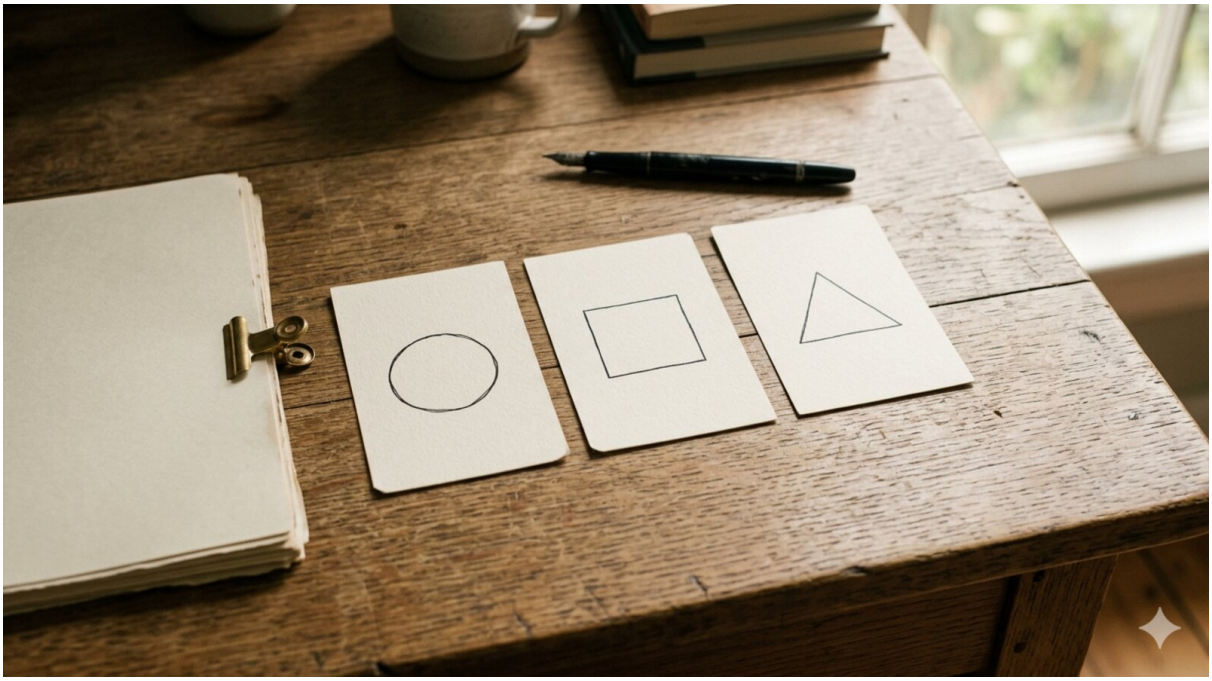


Figure 2: A wooden desk seen from above with three small index cards arranged in a horizontal row, each card showing a hand-drawn symbol, suggesting classification before action.

What is the Problem?

A general-purpose sync daemon bundles concerns that want different treatment:

- **Configuration** wants version history, review before change and the ability to roll back. Git provides all three; whole-file sync approximates none of them.
- **Project content** wants per-project history for code and durable, restorable copies for data. Neither requires that every machine hold a live, mutable replica of everything.
- **Append-only history** wants either one writer or an append-aware merge. Whole-file sync gives neither and produces a conflict copy on every overlap.
- **Settings folders** belong to none of these categories by design, yet a symbolic link can quietly place them in the synced tree. They accumulate credentials that tools create long after the link was made.

The first three items are a question of fit. The fourth is a question of security, and it is the one that changed the course of this migration. A useful test for any file is to ask which of the four it is before asking where it should go.

Prerequisites

- **Machines:** macOS 13 or later, or a recent Linux distribution. Paths in this post follow macOS conventions.

- **Tools:** git, the GitHub CLI (`gh`), and a password manager for any credential that has to be replaced (this series uses `pass`; see Setting Up pass).
- **Backups in place first:** at least one independent backup of every machine before any shared copy is deleted. The backup design is in Research Backup Architecture.
- **Access to the sync account's web interface,** in particular its list of linked devices.
- **Time:** an afternoon to inventory and classify; a day or two to move, verify and uninstall across several machines.

Where Everything Went

Configuration: one dotfiles repository

Shell, editor, git and tool configuration lives in a single git repository at `~/dotfiles`, on a path no sync client touches. An `install.sh` creates the links into `$HOME` and installs the user launchd jobs; `~/bin` on each machine is a folder of links into `~/dotfiles/bin`. The repository also carries a `Brewfile`, so the installed application set is declared rather than remembered, and a list of TeX packages for a per-user TinyTeX installation.

Two rules keep the repository safe. Secrets never enter it, and any tool that writes credentials into its own configuration folder has that file excluded explicitly. The GitHub CLI is the example: it stores its login token in `config/gh/hosts.yml`, which is listed in the repository's `.gitignore` (see 'The Exposure' below for why that line exists).

Project content: local folders, git for code, backups for the rest

`~/prj` is now an ordinary local folder on each Mac. Scripts refer to it through one environment variable, set in `~/.zshenv`, which made the move a two-step change: first every script was rewritten to use the variable while it still resolved to the Dropbox path, then the variable was pointed at the local folder.

```
# ~/.zshenv: the first existing candidate wins, so the same file
# worked before, during and after the move.
if [[ -z "${PRJ_ROOT:-}" ]]; then
  for _candidate in \
    "$HOME/prj" \
    "$HOME/Dropbox/prj" \
    "$HOME/Library/CloudStorage/Dropbox/prj"
  do
    if [[ -d "$_candidate" ]]; then
      export PRJ_ROOT="$_candidate"
      break
    fi
  done
  unset _candidate
fi
```

Code moves between machines through each project's own git remote. Nothing synchronizes the rest of `~/prj` automatically. It is protected instead: nightly Borg backups to the home server cover every machine, and julia and athena also mirror `~/prj` to three cloud providers, each machine into its own destination folder. That design, and the failures it had to survive, belong to Research Backup Architecture. Data that is too large or too shared for this arrangement is the subject of Object Storage for Research Data.

Documents, shared utilities and scratch space

`~/docs` (personal, research and administrative documents, under half a gigabyte) became a plain local folder on each Mac, covered by the Borg backups. The copies are independent; documents are edited on one machine at a time. A folder of shared LaTeX and R utilities was first moved out the same way and later retired altogether once nothing still depended on it. A sandbox folder, whose links had already gone stale on one machine, was deleted rather than migrated.

Local caches of an authoritative source

Some content that looks like data is a cache of something held elsewhere. Mail is the clearest case: the IMAP server holds the mailbox, `mbsync` mirrors it into a local Maildir under `~/.local/share/mail/`, and `notmuch` indexes it. A second machine rebuilds both from the server rather than receiving them by sync. The same reasoning covers package caches, language-server indexes and similar derived state: each machine rebuilds its own, and no sync or backup tier needs to carry it.

Append-only history: per-machine state

Shell history, editor state and AI-assistant prompt logs are now ordinary local files on each machine and are not synchronized at all. The alternative, an append-aware tool such as atuin for shell history, remains available for anyone who wants shared history; the simpler rule was sufficient here. The last conflict copies of the prompt log, dated a week before the move, were found on all three Macs and removed. They cannot recur, because nothing in that folder is synced.

The Exposure: Settings Folders in a Synced Tree

On julia and athena, a settings folder had been a symbolic link into the Dropbox tree for months. Nothing about it was visible in daily use. Every file a tool later wrote into that folder was therefore uploaded to Dropbox's servers, including:

- the GPG private key that protects the password store;
- both machines' Borg backup passphrases;
- the OAuth tokens `rclone` uses for Google Drive and OneDrive;
- an AI service API key;
- stale copies of the password store and of older settings.

There was no way to establish whether anyone else had read that data, so every exposed credential was treated as compromised and replaced. The rotation covered a new GPG key with the store re-encrypted to it, new Borg passphrases, new SSH keys for the code-hosting services, revoked and reissued OAuth authorizations, and new API keys and app passwords. The procedures are in Setting Up pass and Secrets Management for the Workflow Construct. Testing the old credentials mattered as much as issuing new ones: some were already dead, and some were still live until they were revoked.

A second copy surfaced on joe. When macOS moved Dropbox to its newer file-provider integration, it had renamed the old local Dropbox folder and left it in place, and that frozen copy included an old folder of SSH keys and tokens. Every item in it was checked against the service it belonged to (dead, expired, removed, or for a server that no longer exists) before the frozen folder was deleted.

The same hazard exists inside a git repository. The GitHub CLI's settings folder was linked into the dotfiles repository, so its live login token appeared as an uncommitted change on every machine. It was never committed, and the token file is now excluded, but the only difference from the Dropbox case was that this leak was waiting for a commit instead of happening on its own.

The lesson for the migration plan is that the inventory of a synced tree is also an inventory of secrets, and it should be done first. A search of every linked settings folder for keys, tokens and passphrases decides whether the project is a tidy-up or an incident response.

Migration Sequence

The order matters because the backup jobs must stay valid at every step and because a shared copy can only be deleted once.

1. **Inventory and classify.** List every top-level item in the synced folder and every link into it from `$HOME`, hidden folders included. Assign each to configuration, project content, history, settings or 'delete'. Search the settings folders for credentials before anything else.
2. **Back up every machine independently.** Confirm a recent, successful backup that does not depend on the sync client.
3. **Introduce the path variable.** Rewrite scripts to use `$PRJ_ROOT` while it still resolves to the synced path.
4. **Pause every scheduled job that reads what is moving.** A nightly cloud backup that started at 02:00 was still running when the first folder moved and logged thousands of 'file not found' errors. Nothing was lost, because deleted files go to a dated folder rather than away, but every backup job was paused for the rest of the move.
5. **Copy to every machine, verify each copy, then delete the shared copy.** See the next section for why this order is not optional.
6. **Repoint and resume the backups.** Confirm in each job's log, not from its schedule, that it now reads the local folder.
7. **Uninstall the client** after the link audit described below.

Deleting a Shared Copy Safely

Deleting content from a shared sync account is a fleet-wide operation, and three incidents showed what that means in practice.

Emptying propagates. Once julia's copy of the project tree was removed from the account, the removal began to propagate to the other Macs, which were still reading their projects from the synced folder. athena's tree was copied to a local folder minutes before the removal reached it. joe's could no longer be read in place by the time it was checked, so its 19 GB tree was restored by copying directly from julia's verified copy over the LAN.

A forgotten client restores what was deleted. zeus, the Linux laptop, had its own Dropbox client and stale local copies of the project tree and of an old SSH key folder. Nothing on zeus used either, which had seemed reason enough to leave it alone. When its client reconnected, it uploaded both again, with their original dates. The client was removed from zeus and the content deleted a second time.

A partial copy restores part of it. A smaller reappearance followed. The account's own list of linked devices showed a phone that had never been part of the workflow and had cached a selection of files. It was unlinked, along with zeus, which still appeared in the list after its local uninstall, and the remaining content was deleted for the last time.

The procedure that follows from these incidents:

1. Read the account's linked-device list first. It, and not the list of machines one remembers, is the inventory.
2. Unlink every device that is not part of the migration.
3. Copy to every machine that needs the content and verify each copy (size, file count, and for repositories a `git log` that matches).
4. Delete the shared copy last, then check the account again a day later.

Uninstalling the Client

Dragging the application to the Trash removes only the application. The rest has to be found:

- **Links into the synced tree.** Audit `$HOME`, including hidden folders, for symbolic links whose targets lie inside the sync folder. A link that still resolves is something still in use; a link that dangles is debris. Both must be resolved before the uninstall, or some tool fails in an obscure way afterwards.
- **Preference files, caches, login items and helper extensions,** some of them installed system-wide and requiring administrator rights to remove.
- **The file-provider registration** through which macOS presents the cloud folder in Finder. It is normally released by the vendor's uninstaller; Dropbox has none. On two Macs one empty registration folder could not be removed by any method short of reinstalling the client; it holds no files and connects to nothing, and was left in place.

```
# Links in $HOME (one level, hidden included) that point into a
# Dropbox path. Repeat for deeper folders that you know held links.
find "$HOME" -maxdepth 1 -type l -print0 |
  while IFS= read -r -d '' link; do
    target=$(readlink "$link")
    case "$target" in
      *Dropbox*) printf '%s -> %s\n' "$link" "$target" ;;
    esac
  done
```

The audit also found a stale checkout of the dotfiles repository on one Mac whose backup scripts still read the old Dropbox path. That machine's nightly cloud backup had been copying a frozen copy of the project tree for weeks while every run completed normally.

Keeping Machines in Step Afterwards

Nothing synchronizes the machines automatically any more, and that is deliberate. Two habits replace the daemon.

Pull the shared configuration. A change made in `~/dotfiles` on one machine reaches the others only after a `git pull` on each. The shell helps: entering any git repository compares it with its last-fetched upstream and, if it is behind, prints a one-time warning and keeps a small `↓N` indicator in the prompt until the repository is pulled. The underlying fetch is throttled to once every four hours per repository, so the check costs nothing on every prompt.

Authenticate git through the GitHub CLI. Repository remotes use HTTPS, and git obtains credentials from the `gh` credential helper, which keeps its token in the macOS Keychain. This removes the need for a separate SSH key per machine for code hosting. Two details matter. The token needs the `workflow` scope, or GitHub refuses any push that adds or changes an Actions workflow file. And Keychain items are not available to a plain SSH session, so a pull run remotely into a Mac fails where the same command at the keyboard succeeds.

```
# One-time setup on each Mac: log in, then let gh act as git's
# credential helper and default to HTTPS for new clones.
gh auth login --hostname github.com --git-protocol https
gh auth setup-git
gh config set git_protocol https --host github.com
# Add the scope needed to push changes to Actions workflow files.
gh auth refresh --hostname github.com --scopes workflow
```

Project work follows the same discipline: commit and push before leaving a machine, pull before starting on another. For content that is not in git, the rule is one machine at a time per project.



Figure 3: Two open-top boxes side by side on a workbench, the left box full of mixed objects and the right box divided into three sorted compartments, with a single object mid-transfer between them.

Things to Watch Out For

1. **Backup sources go stale silently.** Symptom: a nightly backup completes normally while its source no longer exists or no longer holds the live copy. Borg, for example, skipped `~/prj` and two sibling folders for days after they stopped being links, logging a single warning per run. Fix: name sources directly, check each job's log and file counts after every structural change, and guard scripts with an explicit test that the source exists.
2. **Git working trees inside a synced folder corrupt.** Symptom: phantom changes or a damaged index. Fix: keep every repository on a path no sync client watches.
3. **Links that resolve on one machine dangle on another.** Symptom: a configuration link works on the machine that created it and fails elsewhere. Fix: let the installer create links relative to `$DOTFILES`, never to a literal sync path.
4. **Conflict copies do not announce themselves.** Symptom: files named '(conflicted copy)' accumulate beside the original. Fix: stop syncing append-only files; until then, search for them periodically.
5. **In-place editors can truncate files on cloud-mounted paths.** Symptom: `sed -i` (or `perl -i`, `awk -i inplace`) exits 0 and leaves an empty file, and the `.bak` copy is lost the same way. Fix: never run in-place editors under `~/Library/CloudStorage/`; edit a copy elsewhere and move it back.
6. **Installed launchd jobs are copies.** Symptom: editing the job file in the dotfiles repository changes nothing. Fix: edit the source, rerun the installer, then unload and load the job.
7. **One bad link can stop a mirror's deletions.** Symptom: a cloud mirror grows without bound although every run looks routine. A link pointing to itself made `rclone sync -L` report

a listing error, and on any such error it skips every deletion for the run. Fix: remove loops and dangling links from backed-up trees; details in Research Backup Architecture.



Figure 4: A closed wooden chest with three small brass clasps on a stone surface, an open notebook and a capped fountain pen beside it, suggesting a deliberate and completed reorganization.

What Did We Learn?

Lessons Learned

Conceptual:

- Classification comes before mechanism. Configuration, code, data, history and settings each have a right answer, and they are not the same answer.
- A synced folder is a distribution channel for whatever is written into it. Settings folders linked into it carry credentials created long after the link.
- Per-machine state is often the correct design, not a compromise.

Technical:

- One environment variable gives enough indirection to move a project tree in two safe steps.
- HTTPS remotes with the `gh` credential helper remove per-machine SSH keys for code hosting, at the cost of the Keychain-over-SSH caveat.
- A behind-upstream indicator in the prompt replaces most of what a sync daemon did for configuration, without hiding anything.

Gotchas:

- The device list of the sync account is the only reliable inventory of who can restore deleted content.
- A job that exits normally has not necessarily done its work; read the log.
- A stale checkout of shared scripts on one machine reproduces old bugs there, however long ago they were fixed elsewhere.

Limitations

- The account described here was single-user. A folder shared with collaborators needs agreement on the move and is out of scope.
- The sync account itself remains open; the client is gone, but closing the account is a separate decision.
- Moving to local folders gives up instant cross-machine availability of uncommitted work. The rule of committing before switching machines is a habit, not an enforcement mechanism.
- The fresh-machine bootstrap time has not been measured since the move; a restore runbook and the Brewfile exist, but no timed rebuild has been run.

Opportunities for Improvement

1. Add a `verify` target to the dotfiles repository that checks, on demand, that every link resolves, `$PRJ_ROOT` exists, and each scheduled job's last exit status is zero.
2. Time a full rebuild of one machine from the runbook and record the result.
3. Add a pre-commit secret scanner to the dotfiles repository, so the next token that lands in the working tree is caught by a tool rather than by chance (see Secrets Management for the Workflow Construct).
4. Decide whether to close the sync account, after a final review of its remaining contents and version history.

Wrapping Up

Leaving Dropbox was mostly a matter of deciding what each file was. Once configuration had a repository, projects had local folders and backups, history files stayed on the machine that wrote them, and settings folders were back in `$HOME`, the remaining work was careful copying and a disciplined order of deletion. The parts that cost time were the parts a sync daemon had hidden: credentials in linked folders, devices that still held copies, and backups that kept succeeding at the wrong job.

Main takeaways:

- Inventory secrets before files; a linked settings folder can turn a migration into a credential rotation.
- Copy everywhere, verify, then delete the shared copy, and check the account's device list before trusting that anything is gone.
- Audit `$HOME` links before uninstalling the client.
- Replace the daemon with explicit habits: `git pull`, a prompt that shows when a repository is behind, and HTTPS remotes through the `gh` credential helper.

- Verify every backup by its log after every structural change.

See Also

Security series:

1. Security Foundations for a Multi-Machine Research Fleet
2. Setting Up pass: a Unix Password Manager
3. Secrets Management for the Workflow Construct
4. Migrating Off Dropbox: Beyond Dotfiles (this post)
5. Research Backup Architecture: Borg, Cloud Copies, and Time Machine
6. A Home Server for a Five-Machine Research Fleet
7. Object Storage for Research Data: The Layer Git Cannot Hold
8. Running the Quarterly Security Audit for the First Time: What Actually Broke

External references:

- GitHub CLI manual: `gh auth setup-git`.
- Dropbox account security page, which lists every device ever linked to the account.
- Atuin, append-aware shell history across machines.
- XDG Base Directory Specification, for where per-machine state belongs.

Reproducibility

Component	Version
OS	macOS 26.6 (three Macs); Linux Mint 22 (zeus)
Shell	zsh 5.9
git	2.55
GitHub CLI	2.101
rclone	1.75
Borg	1.4.5
Date verified	2026-10-01

The `PRJ_ROOT` block and the `gh` commands are the ones in use on the fleet. The link-audit loop was written for this post and checked on one Mac; adapt the pattern for other sync clients.

Let's Connect

Questions, suggestions, or corrections are welcome.

- **GitHub:** rgt47
- **Twitter/X:** @rgt47
- **LinkedIn:** Ronald Glenn Thomas

- **Email:** Contact form

I would like to hear from you if:

- You find an error, or a better approach to anything in this series.
- You run a small research fleet and have found where this design breaks.
- You want to compare notes on backups, secrets, or reproducible research infrastructure.